D002

Core Framework Architecture

D002 establishes the backbone of Echo Engine. It defines the framework that every later subsystem depends on: bootstrap, lifecycle, services, events, diagnostics, math, utilities, and extension rules. The document is architecture only. It does not describe runtime, ECS, rendering, physics, or assets.

Field	Summary
Objective	Define the canonical framework layer and the contracts that later milestones must not violate.
Scope	Framework, bootstrap, service registration, state, events, diagnostics, math, utilities, and interfaces.
Non-scope	Runtime D007, ECS, rendering, physics, assets, animation systems, and gameplay systems.
Outcome	A stable dependency-injected framework backbone with clear ownership and freeze criteria.

The purpose of D002 is to make the rest of the project build on a clean spine rather than on ad hoc service lookup, hidden state, or platform-specific assumptions. The framework must remain small in surface area but strong in contract discipline.

1. Framework Mandate

Why the core framework exists and what problem it solves.

The framework is the stable center of Echo Engine. It is not the runtime itself and it is not a feature subsystem. It is the structural layer that provides composition, lifecycle control, shared abstractions, and the minimum set of utilities needed for every other system to behave predictably.

D002 is intentionally strict about boundaries. If the framework becomes a convenience bucket for unrelated code, later milestones will inherit ambiguity. The framework must therefore stay focused on orchestration, not domain logic.

The key architectural goal is separation of concerns. The framework owns the shape of engine execution, while services own behavior, diagnostics own reporting, and utility libraries own reusable helpers. That separation keeps the project testable and makes later milestone reviews much easier.

Framework mandate rules:

• own the bootstrap path, composition root, and engine-level lifecycle state

• provide service registration, service resolution, and service manifest semantics

• offer event dispatch, diagnostics, and performance counters as shared facilities

• remain platform-independent and free of feature-specific assumptions

• avoid hidden global mutable state and avoid implicit service creation

This mandate makes the framework the contract boundary for the engine. Later milestones may extend it, but they should not re-interpret it.

2. Scope and Boundaries

What belongs in src/framework and what must stay outside.

The current directory structure defines a disciplined framework layer: core, diagnostics, events, interfaces, math, services, state, and utilities. Those folders are not arbitrary convenience groupings. They are the architectural compartments that keep the core clean and make ownership obvious.

Anything that describes framework behavior or framework-wide contracts belongs here. Anything that describes feature behavior belongs elsewhere. The framework may expose hooks for features, but it must not absorb feature logic into its own core implementation.

Folder	Role
core/	Bootstrap orchestration, framework identity, builder logic, and composition flow.
services/	Service base types, service manifest contracts, registration policy, and service lifecycle.
events/	Event, EventBus, publish/subscribe policy, and delivery semantics.
state/	Framework state model, lifecycle transitions, and execution phases.
diagnostics/	Performance counters, health reporting, and diagnostic event pathways.
math/	Framework-level math primitives and non-domain numeric utilities.
utilities/	Shared helpers that are framework-safe and do not depend on feature domains.
interfaces/	Public contracts and abstraction boundaries used across framework modules.

The boundaries are deliberate. Core does not reach into feature code. Services do not define their own service locator. Events do not become hidden command channels. Math and utilities do not turn into a dumping ground for unrelated helpers.

3. Dependency Model

Why the framework dependency flow points inward and downward only.

The framework uses a one-way dependency model. Bootstrap leads into Framework, Framework leads into ServiceManifest, ServiceManifest leads into Services, and services use Events, Diagnostics, Math, and Utilities only through contracts that remain stable across milestones. That direction keeps the core easy to reason about.

The reason is architectural safety. The outer orchestration layer should know more about wiring than the inner services know about host details. This prevents cycles, makes initialization deterministic, and allows each layer to be reviewed independently.

Layer	Owns	Reason
Bootstrap	Host entry and startup intent.	Creates the initial composition path.
Framework	Engine-level orchestration and lifecycle.	Owns the framework backbone.
ServiceManifest	Declared services and their metadata.	Defines what can be registered.
Services	Behavioral units with clear contracts.	Execute through framework control.
Events	Publish/subscribe infrastructure.	Carries framework signals.
Diagnostics	Health and performance visibility.	Reports state without coupling.
Math	Shared numeric foundation.	Supports all layers, no domain logic.

This flow exists to keep dependency ownership simple. Higher layers may depend on lower abstractions, but lower layers should never need to know about higher-level composition details. That rule is the difference between a framework and a pile of helpers.

4. Bootstrap and Composition Root

Bootstrap.js, FrameworkBuilder, and CompositionRoot define the entry path.

Bootstrap is the first intentional step. It does not perform application logic. It prepares the engine entry context, selects the composition root, and passes control into the framework with explicit configuration and service declarations.

FrameworkBuilder owns assembly logic. It is the policy object that turns a manifest and a set of host inputs into a usable framework instance. CompositionRoot is the place where concrete dependencies are bound. The two are related, but not the same thing.

Bootstrap.js

Bootstrap is the host-facing entry point for framework startup.

Aspect	Architecture guidance
Purpose	Collect startup inputs and hand them to the framework in a controlled order.
Responsibilities	Validate startup inputs, select the composition root, and trigger framework construction.
Ownership	Owns startup sequencing only, not feature behavior.
Lifecycle	Exists before the framework instance and ends after successful handoff.
Dependencies	Depends on host environment data and the framework contract layer.
Public API	High-level bootstrap method(s), startup context object, and status/result output.
Must never do	Must never instantiate services directly or hide failures behind silent fallback.
Later milestones	Will reuse the same boot contract for runtime, editor, and tooling entry points.

Framework.js

Framework.js is the framework owner and lifecycle coordinator.

Aspect	Architecture guidance
Purpose	Provide the core engine backbone and own the active framework state.
Responsibilities	Initialize services, advance execution phases, dispatch events, and coordinate shutdown.
Ownership	Owns framework state, service registry, and lifecycle transitions.
Lifecycle	Constructed by the builder, initialized once, updated repeatedly, then shut down cleanly.
Dependencies	Depends on service manifest, event infrastructure, diagnostics, and state contracts.
Public API	Initialize, update, shutdown, status snapshot, and service access methods.

Aspect	Architecture guidance
Must never do	Must never create hidden service instances or assume feature-specific behavior.
Later milestones	Will use the same framework instance as the root for runtime, editor, and tool subsystems.

5. Service Manifest and Service Registration

The manifest is the declarative contract for what the framework may host.

The ServiceManifest is the framework?s declaration surface. It tells the framework what services exist, how they are categorized, whether they are required, and what order constraints apply. It exists to prevent registration from becoming hidden procedural code.

Service registration must be explicit. A service manifest can express intent, capability, dependencies, and lifecycle rules, but the framework should still validate each service before activation. That separation keeps the system transparent and reviewable.

ServiceManifest.js

ServiceManifest is the declarative registry of framework services.

Aspect	Architecture guidance
Purpose	Describe which services exist and how they relate before activation.
Responsibilities	Declare identity, requiredness, dependency order, and capability hints.
Ownership	Owns registration intent, not executable behavior.
Lifecycle	Created during bootstrap, consumed during initialization, then treated as read-only.
Dependencies	Consumes FrameworkBuilder inputs and service metadata only.
Public API	Register, inspect, validate, and enumerate service descriptors.
Must never do	Must never instantiate services or mutate active service instances.
Later milestones	Will let runtime, editor, and feature milestones declare their own service sets safely.

Registration policy should prefer explicit ordering over implicit discovery. That choice slightly increases the cost of declaring a service, but it dramatically reduces startup ambiguity and makes failure reports easier to understand.

The service manifest should also be able to express compatibility notes so that future milestones can evolve service contracts without breaking the host silently.

6. Service Base Class and SceneService Relationship

Service is the standard unit of framework-owned behavior.

The Service base class is not a convenience wrapper. It is the standard behavioral contract that says how a framework-managed unit participates in initialize, execute, suspend, and shutdown phases. It exists so that every service follows the same lifecycle vocabulary.

SceneService is a special relationship example because later scene systems are likely to sit above the framework but still need to plug into its lifecycle. D002 must define that relationship as a dependency boundary, not as a feature promise.

Service base class

Standard lifecycle contract for framework-managed behavior.

Aspect	Architecture guidance
Purpose	Give every service a common lifecycle and ownership model.
Responsibilities	Handle initialization, update participation, cleanup, and state exposure.
Ownership	Owns the service?s lifecycle state and local invariants.
Lifecycle	Constructed by composition, initialized by framework, updated by framework, destroyed by framework.
Dependencies	Depends on framework contracts, events, diagnostics, and explicit injected services.
Public API	Initialize, update, shutdown, status, and optional capability declaration.
Must never do	Must never hide asynchronous side effects or self-register globally.
Later milestones	Will be subclassed or adapted by runtime, scenes, editor, and tool services.

SceneService relationship

Scene integration is a consumer of framework services, not a framework exception.

Aspect	Architecture guidance
Purpose	Define how scene-oriented systems attach without taking over the framework.
Responsibilities	Expose scene lifecycle hooks through framework services and events.
Ownership	Scene behavior owns scene logic; framework only owns hosting rules.
Lifecycle	Exists after framework startup and must cleanly detach on framework shutdown.
Dependencies	Depends on service base contracts, event bus, and framework state.
Public API	High-level scene attach, detach, and status methods only.
Must never do	Must never become a hidden ECS or runtime substitute inside the framework layer.

Aspect	Architecture guidance
Later milestones	Will bridge to scene management, content loading, and gameplay systems later.

7. Framework Lifecycle, State Management, and Update Flow

Construction, initialization, registration, execution, shutdown, and cleanup are explicit phases.

D002 defines a complete lifecycle because the framework must be predictable under load, during failure, and during shutdown. The lifecycle is not just a sequence of methods. It is the engine-level contract for when services may register, when they may execute, and when they must stop.

State management must be explicit and inspectable. Framework state is a summary of which phase the system is in, which service groups are active, and whether the framework is healthy enough to continue. If state is ambiguous, the framework becomes difficult to debug and impossible to freeze cleanly.

Phase	Framework rule
Construction	Objects are created from injected dependencies and configuration only.
Initialization	The framework validates the manifest, registers services, and prepares shared facilities.
Registration	Services are categorized, ordered, and bound to the lifecycle.
Execution	The update loop advances services, dispatches events, and records diagnostics.
Shutdown	Services are stopped in reverse-safe order and told to release resources.
Cleanup	Framework-owned state is cleared and the final status snapshot is emitted.

Update flow must remain framework-owned even when later milestones define their own execution semantics. D002 should expose a framework update entry point, but it must not make assumptions about frame pacing, simulation rules, or feature-specific timing behavior.

FrameScheduler is therefore only a compatibility wrapper in D002. It exists to preserve a stable entry name for future timing models without allowing timing policy to leak into the core architecture.

FrameScheduler

Compatibility wrapper around framework update sequencing.

Aspect	Architecture guidance
Purpose	Offer a stable adapter name for future timing and tick models.
Responsibilities	Translate update requests into framework execution phases.
Ownership	Owns scheduling compatibility, not time policy.
Lifecycle	Created by framework composition and retired when the framework shuts down.
Dependencies	Depends on Framework, state, and diagnostics only.
Public API	Schedule/update/advance-style entry points at a high level.
Must never do	Must never define the real runtime clock or domain simulation policy.

Aspect	Architecture guidance
Later milestones	Will be replaced or specialized by the runtime milestone without breaking the framework.

8. Event Infrastructure

Event and EventBus provide the framework-wide signal path.

Events are the framework?s internal language for broadcast-style coordination. They should be lightweight, typed conceptually, and easy to inspect. They are not a substitute for service APIs. They exist to decouple producers from consumers without hiding ownership.

EventBus is the delivery mechanism. It is responsible for routing, subscription management, and controlled delivery semantics. It should stay framework-centered and not become a general-purpose application message broker inside the core.

Event

Event is the atomic framework signal envelope.

Aspect	Architecture guidance
Purpose	Describe a framework event in a stable, inspectable form.
Responsibilities	Carry event identity, source, timestamp/context, and payload metadata.
Ownership	Owns the signal shape, not the subscriber list.
Lifecycle	Created by a producer, delivered by the bus, then treated as immutable.
Dependencies	Depends on interfaces, diagnostics, and optionally math/utilities for payload support.
Public API	High-level event identity and payload accessors only.
Must never do	Must never self-route, mutate after publish, or hide side effects.
Later milestones	Will carry runtime, editor, or gameplay signals through the same envelope model.

EventBus

EventBus routes events across the framework boundary.

Aspect	Architecture guidance
Purpose	Provide publish/subscribe infrastructure for framework-level signals.
Responsibilities	Register handlers, publish events, isolate delivery failures, and support teardown.
Ownership	Owns subscriptions and delivery policy.
Lifecycle	Constructed during framework startup, used throughout execution, drained on shutdown.
Dependencies	Depends on Event, diagnostics, state, and service contracts.
Public API	Subscribe, unsubscribe, publish, dispatch, and clear operations.
Must never do	Must never become a hidden global event singleton or feature bus.

Aspect	Architecture guidance
Later milestones	Will carry runtime notifications, editor signals, and subsystem alerts.

The event layer should remain narrow. If every service starts publishing arbitrary free-form messages, the framework loses its ability to reason about dependencies and failure paths. The bus should therefore enforce discipline rather than act as a catch-all shortcut.

9. Diagnostics Foundation

PerformanceCounters and diagnostic services keep the framework observable without coupling it to a logger implementation.

Diagnostics is a first-class framework concern. The goal is not to hard-code a logging backend. The goal is to make framework health visible through stable counters, structured status snapshots, and a shared diagnostic vocabulary that later systems can consume.

PerformanceCounters sit in this layer because they are part of the framework?s self-knowledge. They should track lifecycle costs, service counts, event throughput, and shutdown behavior at a high level without exposing implementation detail as policy.

PerformanceCounters

PerformanceCounters provide framework-level health signals.

Aspect	Architecture guidance
Purpose	Capture coarse-grained operational metrics for the framework backbone.
Responsibilities	Track initialization time, service counts, event load, and shutdown duration.
Ownership	Owns reported counter values, not feature telemetry.
Lifecycle	Created with the framework, sampled during execution, emitted in status snapshots.
Dependencies	Depends on state, events, and math for measurement convenience only.
Public API	Read-only access to counter snapshots and summary values.
Must never do	Must never become a profiler or a feature analytics store.
Later milestones	Will be read by runtime, editor, and tooling health views.

Topic	Rule
Diagnostics guidance	Diagnostics should explain what failed, where it failed, and which layer owns the failure.
Separation rule	Logging is an output channel, not the architecture itself.
Signal discipline	Framework warnings should be structured enough to support automation later.
Status rule	Each phase should be able to produce a concise health snapshot.

When later milestones request richer telemetry, the framework should extend the diagnostic contract rather than overwrite it. That is the key to long-term compatibility.

10. Mathematical Foundation

The framework math library provides the shared numerical base without pulling in domain behavior.

Framework math exists to keep common numeric operations, vectors, ranges, interpolation helpers, and comparison utilities centralized. It should be small, explicit, and predictable. The math layer is a shared foundation, not a simulation engine.

This is an important architectural distinction. The framework math library must not grow into a domain-specific physics package or a rendering math farm. Later milestones may consume it, but they should not have to fear hidden policy inside it.

Framework Math library

Framework Math is the shared low-level numeric foundation.

Aspect	Architecture guidance
Purpose	Provide reusable numeric primitives and helpers that are safe for the whole engine.
Responsibilities	Host math types, numeric helpers, comparison helpers, and stable value semantics.
Ownership	Owns reusable numeric abstractions only.
Lifecycle	Available as a pure library from the start of framework construction.
Dependencies	Depends on no feature systems; may only depend on language and interface contracts.
Public API	Vector-like, range-like, and utility-style math types at a high level.
Must never do	Must never contain physics policy, game rules, or rendering policy.
Later milestones	Will be used by runtime, rendering, audio, AI, and tooling without redefinition.

The math layer should prefer explicit value semantics and simple names. The framework should not invent multiple parallel types for the same concept unless there is a clear ownership reason.

High-level rule: if a math helper starts to look domain-specific, it probably belongs in a later milestone instead of in the framework core.

11. Utilities Foundation

Utilities are the reusable helpers that do not justify a service or domain subsystem.

The utilities folder should contain only small, stable helpers that are safe to use throughout the framework. This layer exists to avoid duplicating basic functionality, not to absorb design uncertainty. It should stay deliberately boring.

The more reusable a utility looks, the more important its contract discipline becomes. A utility with hidden side effects or unclear ownership can quietly poison several other framework modules, so the bar for inclusion should be high.

Framework utilities

Utilities provide low-risk helpers shared across the framework.

Aspect	Architecture guidance
Purpose	Offer reusable helpers for strings, paths, guards, and framework-safe convenience logic.
Responsibilities	Remain small, pure where possible, and easy to test in isolation.
Ownership	Owns helper behavior only, not engine orchestration.
Lifecycle	Available wherever the framework needs them, with no hidden startup ordering.
Dependencies	Depends on no framework state except through explicit parameters.
Public API	Small utility helpers with clear names and narrow contracts.
Must never do	Must never replace a service, state manager, or policy object.
Later milestones	Will remain a shared support layer as the engine grows.

Utilities should be reviewed with particular care because they tend to proliferate. If a helper becomes large or stateful, it should be promoted into a dedicated service or contract layer.

12. Interfaces and Extension Mechanism

Interfaces define the boundaries; extension points let later milestones plug in without rewriting the core.

The interfaces layer is the framework?s public philosophy made concrete. It should define abstract contracts, not accidental abstractions. Every interface should exist because it represents a meaningful ownership boundary, lifecycle boundary, or integration boundary.

Extension points must be intentional. The framework should allow new services, new diagnostics, new event producers, and future adapters without forcing the core to know feature details. That is how the architecture stays open for extension but closed for inappropriate modification.

Interfaces

Interfaces are the framework?s stable abstraction boundary.

Aspect	Architecture guidance
Purpose	Define what can be depended on without binding callers to concrete classes.
Responsibilities	Describe framework contracts for services, state, events, diagnostics, and helpers.
Ownership	Owns abstraction shape only.
Lifecycle	Created before implementation details, used throughout the engine lifetime.
Dependencies	Depends on the framework?s vocabulary and nothing more.
Public API	Named interfaces, abstract methods, and capability contracts.
Must never do	Must never become a dumping ground for half-formed abstractions.
Later milestones	Will let runtime, editor, and feature modules integrate through stable seams.

Extension mechanism rules:

• new services should register through the manifest and composition root

• new event producers should publish through the event bus contract, not a private channel

• new diagnostics should appear as structured additions, not incompatible replacements

• new utility helpers should remain framework-safe and narrow in scope

The extension mechanism must protect the core from surprise. If the framework can be modified only by editing internal classes, it is not truly extensible.

13. Extensibility and Future Milestone Integration

How later milestones plug into D002 without violating the framework contract.

D002 is designed to be a foundation for the rest of the project. Later milestones should be able to depend on the framework without negotiating the basics again. That means every new subsystem must hook into the framework through defined seams: services, events, diagnostics, interfaces, and the composition root.

The rule is simple. If a later milestone needs a new behavior, it should ask the framework for a new extension point or a new service contract. It should not bypass the framework with a hidden global, a private singleton, or an ad hoc call chain that breaks the ownership model.

Milestone	How it plugs in	Constraint
Assets	Register asset-facing services through manifest and service contracts.	Framework stays asset-agnostic.
Runtime	Use the framework for lifecycle and service hosting only.	Timing policy stays in D007.
Animation	Attach through services and events, not through core rewrites.	Animation remains modular.
Rendering	Consume framework services without relocating framework ownership.	Rendering does not become the core.
Physics	Integrate through service contracts and diagnostics.	Physics stays independent.
AI	Use events and services for coordination.	AI owns behavior, not framework policy.
Gameplay	Build on services and interfaces instead of bypassing them.	Gameplay remains feature-layer logic.
Editor	Treat the framework as a hostable backbone for tooling.	Editor integration stays platform-aware but core-safe.
Networking	Use framework diagnostics and events for state visibility.	Network stack remains separate.

This extension model is intentionally boring. The best framework is the one that keeps later milestones from inventing their own infrastructure because the existing one was too difficult to use.

The architectural tradeoff is simple: the framework gives up some improvisational freedom in exchange for long-term consistency and easier maintenance.

14. Architecture Decisions and Tradeoffs

Why these choices were made and what common mistakes they avoid.

D002 intentionally chooses explicit dependency injection over implicit discovery. That decision adds some setup cost, but it pays off by making ownership visible. It also avoids the common mistake of mixing service location with service registration, which usually turns into hidden coupling.

The framework also favors one class per file and private state by default because the codebase should be readable to future maintainers. A framework should feel like infrastructure, not a maze of utility patterns and nested helpers.

Decision	Why it matters
SOLID	Each class and interface should serve one clear purpose and remain open to extension.
Dependency Injection	Dependencies arrive from composition, not from hidden global state.
Composition Root	All concrete wiring happens in one deliberate place.
Single Responsibility	Framework classes own framework work only, not feature behavior.
Open Closed	The framework should accept new services and adapters without rewriting the core.
Module isolation	Each folder should own a single architectural concern.
Loose coupling	Consumers depend on abstractions and manifests, not implementation shortcuts.
High cohesion	Related behavior stays together so the system remains understandable.
ES Modules	Module boundaries should be explicit and import graphs should remain stable.
No global mutable state	Shared state must be deliberate, not incidental.
One class per file	Classes remain easy to find, review, and freeze independently.
Private fields	Internal state should not become part of the public contract unless approved.

Common mistakes to avoid include overusing service locators, mixing framework policy with feature policy, exposing mutable shared state, and letting convenience helpers bypass the composition root. Each of those mistakes may seem small at first, but they quickly erode the clarity D002 is trying to establish.

15. Verification, Review, and Freeze Criteria

How to confirm D002 is ready to become the canonical framework reference.

D002 should be frozen only when the team can explain the framework from its contracts rather than from its implementation. The verification step checks whether the framework layer is understandable, stable, and safe for later milestones to depend on.

The checklist is intentionally practical. If the architecture cannot pass these review gates, then it is not yet safe to build the rest of Echo Engine on top of it.

Architecture verification checklist

- framework ownership is distinct from service ownership
- bootstrap, framework builder, and composition root have non-overlapping responsibilities
- service manifest and service registration are explicit and deterministic
- framework lifecycle phases are named and ordered
- events, diagnostics, math, and utilities are all in the framework and not in feature code
- no global mutable state is required for normal operation
- the dependency graph flows in the documented direction

Freeze checklist

- each component has purpose, responsibilities, ownership, lifecycle, dependencies, API, never-do, and future interaction clearly stated
- D002 does not describe runtime implementation details or feature systems
- service and state contracts are stable enough for downstream milestones to consume
- extension points are documented and use the framework seam model
- common implementation mistakes are called out and understood

Review checklist

- architecture reviewers can trace how a host reaches the framework through bootstrap
- reviewers can explain where services are registered and why they are not self-owned
- reviewers can describe where event, diagnostics, math, and utility dependencies stop
- reviewers can identify what later milestones are allowed to change and what they are not allowed to change

D002 is ready to freeze when the framework can be summarized as a stable, dependency-injected backbone with a closed set of responsibilities and open extension seams. At that point, later implementation work should conform to the document rather than reinterpret it.